

FractalPulse: Physics-Informed TinyML for Real-Time Cardiac & Respiratory Anomaly Detection

1. Executive Summary

FractalPulse is an ultra-low-power wearable health monitoring system designed for continuous, on-device detection of cardiac, respiratory, and motion-related anomalies. The system fuses ECG, PPG, and accelerometry data, then applies a compressed physics-informed TinyML pipeline to detect early warning patterns such as arrhythmias, respiratory distress, and fall-risk events in real time.

The platform is built for real-world edge deployment: it runs directly on a low-power microcontroller, avoids cloud dependence, preserves individual privacy, and triggers local alerts within milliseconds when clinically relevant anomalies are detected.

This project aligns directly with the theme of Edge AI for Continuous Healthcare Monitoring by combining biomedical sensing, machine learning, and embedded optimization in a wearable form factor.

2. Problem Statement

Modern wearable healthcare devices often struggle with one or more of the following constraints:

- They rely on cloud connectivity and cannot operate reliably in low-bandwidth or offline settings.
- They consume too much power for always-on monitoring.
- They do not combine multiple physiological signals into a unified anomaly model.
- They lack actionable local decision-making for emergency events.
- They create privacy concerns by transmitting sensitive health data externally.

In real-world care scenarios, continuous monitoring must be both low-power and immediate. Delayed detection of cardiac irregularities or respiratory distress can increase risk and reduce intervention effectiveness.

3. Solution Overview

FractalPulse is a wearable edge-AI device that continuously monitors the user using a compact sensor suite and an embedded machine learning engine. The core idea is to transform raw biomedical signals into compact physiological features and then classify abnormalities using a tiny, optimized model that runs entirely on-device.

Core capabilities

- Continuous ECG and PPG monitoring
- Respiratory rate and breath irregularity detection
- Motion and fall detection via accelerometry
- Real-time anomaly scoring and event classification
- Local alerting via haptic vibration, buzzer, or BLE notification
- Secure, private, edge-only processing without cloud transmission

4. Target Use Cases

Cardiac monitoring

- Detection of atrial fibrillation and irregular rhythm episodes
- Early warning for tachycardia/bradycardia events
- Continuous heart-rate variability tracking

Respiratory monitoring

- Respiratory distress detection
- Apnea-like patterns or abnormal breathing rhythms
- Recovery monitoring after exertion

Motion and safety monitoring

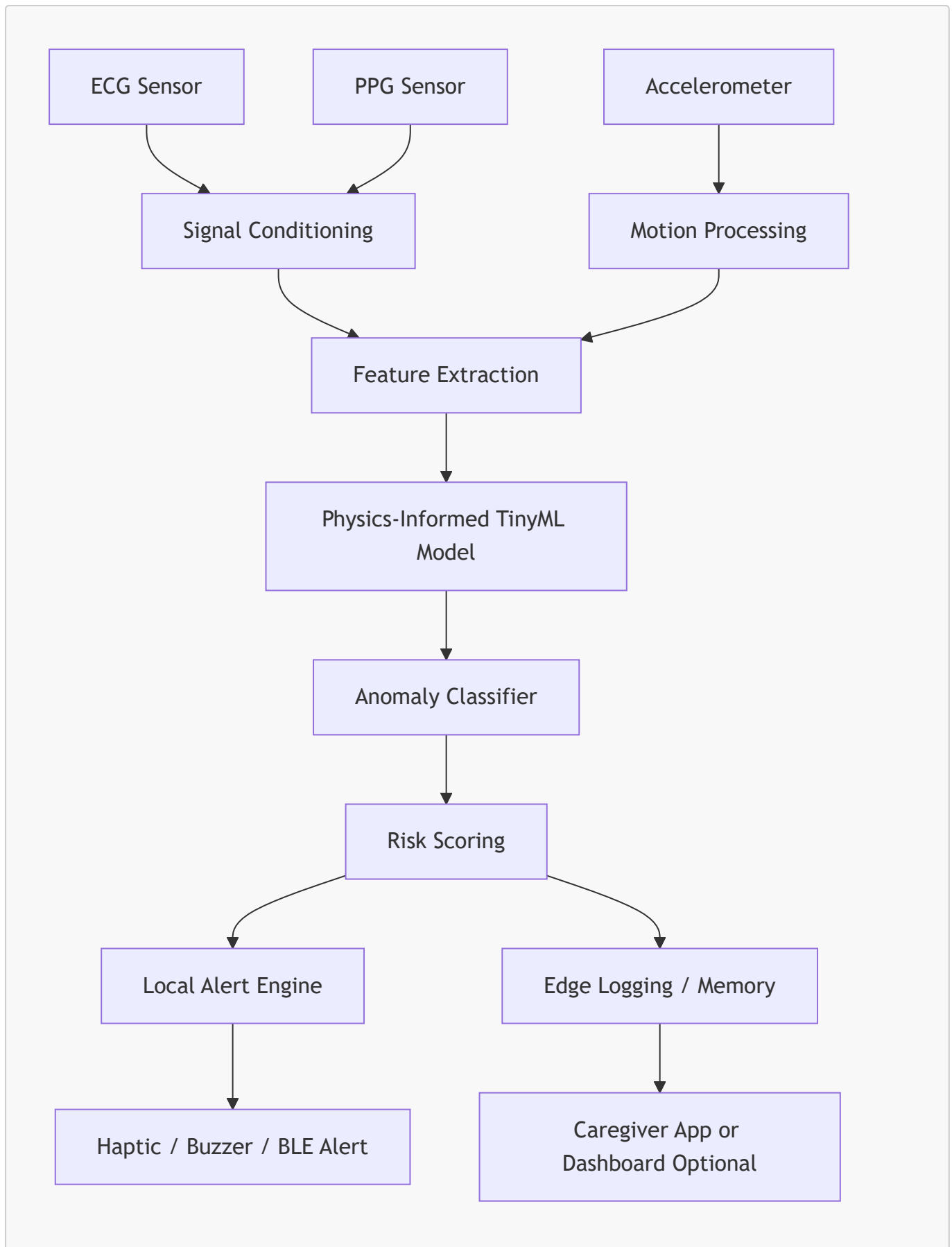
- Fall detection
- Sudden activity changes
- Gait anomalies or inactivity emergencies

Remote care support

- Always-on monitoring for elder care, post-discharge patients, and chronic disease management
 - Low-friction monitoring in home or mobile environments
-

5. System Architecture

High-level architecture



Functional modules

1. Sensor Front End

- ECG electrode interface or analog front end

- PPG optical sensor
- 3-axis accelerometer
- Sensor signal conditioning and filtering

2. Signal Acquisition Layer

- ADC sampling and preprocessing
- Digital filtering for noise suppression
- Synchronization of multi-modal streams

3. Feature Engineering Layer

- R-peak detection for ECG timing features
- Heart-rate variability metrics
- Pulse amplitude and pulse transit characteristics
- Breathing rate estimates
- Signal energy, orientation, and impact features from motion data

4. TinyML Inference Engine

- Quantized lightweight CNN / MLP / temporal model
- Physics-informed loss terms or feature constraints based on known physiological priors
- Runtime inference on microcontroller with minimal memory footprint

5. Decision and Alert Layer

- Risk thresholds for arrhythmia, respiratory distress, and falls
- Event-triggered alert logic
- Local activation of vibration or audible warning

6. Storage and Edge Logging

- Short-term buffering of critical events
- Secure local storage of anomaly summaries
- Optional BLE syncing to a companion app when enabled

6. Physical Hardware Architecture

Recommended hardware stack

- MCU: STM32L4/L5 or Nordic nRF5340
 - Optimized for ultra-low power operation
 - Sufficient compute for TinyML inference
 - Integrated BLE support for optional communications
- Biosensors:
 - AD8232 or similar ECG analog front end
 - MAX30101 / MAX30102 for PPG
 - IMU such as MPU6050 or LIS3DH for motion sensing

- Power management:
 - Li-ion battery with power-efficient PMIC
 - Sleep states and wake-on-event scheduling
 - Duty-cycled sampling to extend wearable battery life

Why this hardware fits the challenge

- MCU choice enables edge-only AI inference with manageable power draw.
 - Analog sensor front ends are mature, compact, and widely used in wearable health devices.
 - BLE or local notification options allow alerting without sacrificing privacy or battery efficiency.
-

7. AI / ML Methodology

Signal fusion strategy

The system combines three complementary streams:

- ECG: cardiac rhythm, conduction abnormalities, irregular beat timing
- PPG: pulse waveform and peripheral perfusion changes
- Accelerometry: motion, posture, impact, and fall events

These streams are synchronized and processed together to reduce false positives and improve classification reliability.

Physics-informed modeling

A physics-informed design is used by encoding physiological priors into the learning pipeline. Examples include:

- Heart rate should remain within plausible ranges over short windows
- Respiration and motion signals should exhibit temporal continuity
- ECG and PPG peaks should maintain realistic timing constraints
- Sudden impact patterns may correlate with falls or high-risk motion states

These constraints improve robustness in noisy real-world wearable conditions and reduce the need for large datasets.

Model types considered

- Tiny CNN for time-series pattern extraction
- Temporal MLP for compact feature-based inference
- Lightweight LSTM or GRU variant for short-window sequence modeling
- Hybrid model combining handcrafted features + tiny neural classifier

Model compression approach

- Quantization-aware training
- Weight pruning
- Fixed-point arithmetic

- Feature reduction to a minimal optimized vector
 - Runtime inference using TensorFlow Lite for Microcontrollers or CMSIS-NN optimized kernels
-

8. Detailed Workflow

Step 1: Sensing

The wearable continuously acquires ECG, PPG, and accelerometer data through analog and digital acquisition interfaces. Sampling is scheduled intelligently to conserve power.

Step 2: Preprocessing

Each signal is filtered and normalized:

- Band-pass filtering for ECG
- DC removal and baseline correction for PPG
- Noise filtering for motion artifacts
- Axis calibration and gravity removal for accelerometer data

Step 3: Feature Extraction

The embedded firmware extracts a compact set of clinically meaningful features, including:

- HR and HRV metrics
- Pulse amplitude and pulse interval features
- Respiratory rate and breathing irregularity indices
- Peak-to-peak variability
- Impact magnitude, jerk, orientation change, and fall-specific motion signatures

Step 4: On-device inference

The feature vector flows into a small, quantized ML model running on the MCU. The model classifies the current state into categories such as:

- Normal
- Arrhythmia risk
- Respiratory distress
- Fall or impact event
- Motion anomaly

Step 5: Decision and alerting

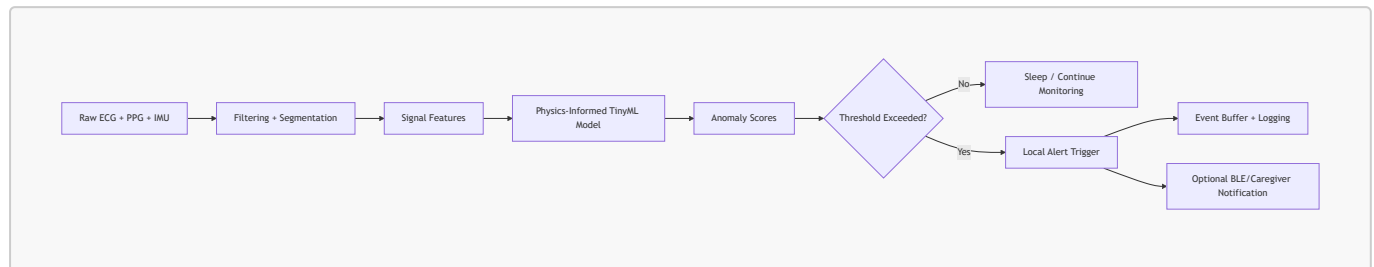
If the anomaly score exceeds a threshold, the device triggers a local decision pipeline:

- Vibration motor engages
- Buzzer or audio cue is sounded
- BLE alert is sent to caregiver app if connected
- Recent event summary is stored for later review

Step 6: Event logging

Short-term local logging captures timestamps, confidence scores, and the signal context around the event. This enables later diagnostic review without continuously uploading raw sensitive data.

9. Data Flow Diagram



10. Embedded Software Stack

Firmware and edge stack

- C/C++ firmware for low-level sensor control
- STM32 HAL / Nordic SDK for hardware abstraction
- CMSIS-DSP and CMSIS-NN for efficient signal processing and inference
- TensorFlow Lite for Microcontrollers for model deployment
- Edge Impulse or custom model export pipeline for rapid prototyping

Deployment process

1. Capture multi-modal health data from sensor hardware.
2. Label clinically relevant events.
3. Train and compress the model.
4. Convert model to TFLite Micro / C arrays.
5. Flash firmware to the MCU.
6. Validate in-device performance under realistic conditions.

11. Power and Performance Considerations

Why it is suitable for wearable deployment

The design targets a battery life that supports continuous daily monitoring, not just short bursts. This is achieved by:

- Duty-cycled sampling for non-critical periods
- Event-triggered wakeups for high-frequency processing
- Lightweight inference model with small memory footprint
- Minimal radio usage to conserve power
- Local edge-only processing to avoid continuous data transmission

Expected power strategy

- Sensor front end: low-power analog block

- MCU sleep mode: background idle state
- Periodic inference window: short active bursts
- Alert channel: only when threshold conditions are met

This helps maintain a practical form factor for daily wearable use.

12. Safety, Privacy, and Clinical Relevance

Data privacy

FractalPulse keeps patient data on-device. It does not require continuous cloud upload, which reduces privacy risk and compliance concerns in sensitive healthcare settings.

Real-time intervention

Because the model runs locally, detection latency is minimized. This is critical for:

- seizure-like episodes
- fall events
- acute respiratory distress
- sudden arrhythmia episodes

Medical relevance

The proposed platform is not a replacement for clinical diagnosis, but it is a strong continuous monitoring and early-warning system that flags anomalies for follow-up or immediate caregiver attention.

13. Competitive Advantages

- Edge-only intelligence reduces latency and network dependence
 - Multi-signal fusion increases robustness and reduces false alarms
 - Physics-informed constraints improve realism and interpretability
 - Ultra-low-power MCU target is suitable for long-term wearable use
 - Privacy-first architecture avoids sending raw patient signals to the cloud
-

14. Key Risks and Mitigations

Risk: motion artifacts corrupt biosignals

Mitigation: signal filtering, sensor fusion, and motion-aware anomaly gating.

Risk: high false positive rate

Mitigation: threshold tuning, confidence-scoring, and multi-signal validation.

Risk: compact model underfits complex physiology

Mitigation: use handcrafted features and small hybrid neural models for strong performance under resource constraints.

Risk: battery life constraints

Mitigation: duty-cycling, event-driven wakeups, and low-power MCU selection.

15. Demo / Prototype Story

A working prototype could include:

- A chest or wrist-worn wearable housing with ECG/PPG sensors and IMU
- A microcontroller board running the TinyML model
- Real-time LED or vibration alerts
- Serial dashboard showing classified states, confidence, and thresholds
- A mobile companion app optional for settings and event review

This makes the concept demonstrable at a hackathon and easy to explain to judges.

16. Project Timeline (Example 2-Week Hackathon Plan)

Phase 1: Sensor and signal acquisition

- Integrate ECG, PPG, and IMU sensors
- Build filtered data collection pipeline

Phase 2: Feature extraction and labeling

- Capture sample datasets and define event labels
- Build feature extraction logic

Phase 3: TinyML model development

- Train compact classifier
- Apply quantization and optimization

Phase 4: Edge deployment

- Port model to MCU firmware
- Validate latency and memory footprint

Phase 5: Alert system and demo

- Add vibration/buzzer alerts
 - Build simple dashboard or device demonstration
-